

Generative Models vs Traditional AI-Driven Anomaly Detection in Large-Scale System Logs: A Comparative Study

Muhammad Asad Mahmood Khan
Department of Creative Technologies
Air University
Islamabad, Pakistan
asadkhanofficial2341@gmail.com

Abstract—The escalating scale and complexity of distributed computing architectures—Hadoop (HDFS), Apache Spark, and Hadoop/OpenStack—have rendered manual and rule-based log monitoring intractable. This paper presents a comparative study evaluating Traditional Machine Learning (ML), sequence-based Deep Learning (DL), and Generative AI models for automated anomaly detection in massive, unstructured log collections. Benchmarking reveals that traditional supervised algorithms (XGBoost, Random Forest) achieve near-perfect precision on structured data but rely on labor-intensive manual feature engineering, while unsupervised baselines such as Isolation Forest exhibit low efficacy on high-dimensional sequences. Supervised deep sequence models (LSTM, GRU, CNN) match this accuracy while removing manual feature extraction, yet degrade sharply under the noise and concurrency of Spark logs. Generative architectures (LSTM-Autoencoder, VAE, GAN), trained strictly on normal data, achieve high ROC-AUC but consistently lower F1-scores, offering a baseline capability for zero-day anomaly discovery rather than outright superiority. Models were independently trained and tested within isolated dataset boundaries (HDFS, Spark, Hadoop) to avoid cross-system contamination. Results confirm that Traditional ML and supervised Deep Learning outperform Generative AI across all evaluated datasets, while highlighting precision trade-offs in highly concurrent environments such as Spark.

Index Terms—anomaly detection, system logs, machine learning, deep learning, generative AI, LSTM, variational autoencoder, generative adversarial network, HDFS, Apache Spark

I. INTRODUCTION

System logs are the primary source of truth for diagnosing failures, tracking execution flow, and ensuring security in modern cloud infrastructures. In distributed computing paradigms such as the Hadoop Distributed File System (HDFS) and Apache Spark, a single transaction can cascade into execution traces spanning thousands of concurrent nodes, generating terabytes of heterogeneous, unstructured log data.

Manual inspection and static, rule-based heuristics have become fundamentally obsolete given this scale. Consequently, the industry has shifted toward Artificial Intelligence—spanning Traditional Machine Learning (ML), Deep Learning

(DL), and Generative AI—to parse, embed, and analyze massive log collections for automated anomaly detection.

A. Problem Statement

Current industry standards for automated log anomaly detection are paralyzed by a dichotomy between accuracy and adaptability. Supervised models such as SVM and XGBoost achieve high accuracy in closed-world scenarios but depend on rigid manual feature engineering (e.g., Event Count Matrices) and labeled datasets, failing under concept drift. Unsupervised baselines (Isolation Forest, One-Class SVM) avoid labeling requirements but struggle to capture temporal and sequential dependencies, producing high false-positive rates and poor detection of unknown, zero-day errors. A unified framework combining supervised accuracy with unsupervised flexibility remains absent.

B. Objectives and Research Questions

This research benchmarks Traditional ML, sequence-based DL, and Generative AI on HDFS, Spark, and Hadoop datasets, producing a tri-fold comparative performance matrix. The core research questions are:

RQ1: To what extent can Generative AI maintain consistent anomaly detection performance across heterogeneous distributed systems compared to Traditional Supervised models?

RQ2: Can Generative Deep Learning eliminate the dependency on manual feature engineering while maintaining high F1-scores?

RQ3: Can deep generative models significantly outperform traditional unsupervised baselines in detecting unknown zero-day anomaly patterns?

II. LITERATURE SURVEY

A. Traditional Machine Learning

Foundational log anomaly detection studies relied on supervised algorithms such as Decision Trees, SVM, Random Forest, and XGBoost, achieving high accuracy on structured, well-balanced datasets after parsing raw logs into fixed templates (e.g., using Drain [3]) and constructing Event Count Matrices or TF-IDF features. These discriminative models are brittle: a shift in log syntax or system architecture can catastrophically degrade performance. Unsupervised baselines

such as Isolation Forest [7] mitigate the need for labeled data but struggle with high-dimensional sequential data, producing high false-positive rates.

B. Deep Learning Sequence Models

To address manual feature engineering, DeepLog [4] modeled system logs as natural-language sequences using LSTM networks, learning normal patterns and detecting anomalies as deviations, outperforming traditional data-mining methods. LogAnomaly [2] extended this by integrating sequential and semantic log information. CNN-based spatial models have also been adapted to detect local anomalies within log windows. These supervised DL models remain vulnerable to the out-of-vocabulary problem, where unseen log events cause misclassification of normal operations.

C. Generative and Transformer Models

Recent work leverages self-supervised and Transformer-based architectures such as LogBERT to capture contextual relationships between log events. Generative architectures—VAEs, GANs (e.g., LogGAN), and LSTM-Autoencoders—are trained exclusively on normal operational data, flagging anomalies via reconstruction error or probabilistic deviation. While these models theoretically remove dependency on labeled data and feature engineering, they are computationally expensive and sensitive to reconstruction thresholds, degrading in highly concurrent, noisy environments such as Apache Spark.

D. Research Gap

Most existing studies evaluate models under isolated, single-dataset benchmarks (typically HDFS), which fails to reflect complex enterprise environments. Comprehensive, multi-source comparative research subjecting Traditional ML, DL, and Generative AI to identical conditions across structurally distinct datasets—HDFS, Spark, and Hadoop—remains lacking, leaving the trade-off between accuracy and architectural autonomy largely unresolved.

III. MATHEMATICAL FORMULATION

Raw, unstructured logs are parsed into structured event templates. A parsed log sequence of length T is defined as an ordered set of log events:

$$E = \{e_1, e_2, \dots, e_T\} \quad (1)$$

where each event $e_t \in \mathbb{R}^d$ is a d -dimensional embedding of the log event at time step t , bypassing manual Event Count Matrices.

For sequence modeling, an LSTM network updates its hidden state h_t from event e_t and prior state h_{t-1} , predicting the probability of the next event among vocabulary V :

$$P(e_{t+1} = v_i | e_1, \dots, e_t) = \exp(W_i h_t + b_i) / \sum_j \exp(W_j h_t + b_j) \quad (2)$$

Generative models are trained strictly on normal sequences within isolated dataset boundaries. A Variational Autoencoder (VAE) learns the underlying distribution of normal sequences by optimizing the Evidence Lower Bound (ELBO):

$$LVAE = -Eq\phi(z|x)[\log p\theta(x|z)] + DKL(q\phi(z|x) \parallel p(z)) \quad (3)$$

where x is the input log sequence, z the latent representation, $q\phi(z|x)$ the encoder, $p\theta(x|z)$ the decoder, and DKL the Kullback–Leibler divergence regularizer.

At test time, an anomaly score $A(x)$ is computed from the reconstruction loss; an anomaly is flagged when this loss exceeds a dynamic threshold τ :

$$A(x) = \|x - p\theta(x|z)\|^2 > \tau \quad (4)$$

IV. METHODOLOGY

The study follows a five-phase comparative methodology (Fig. 1), independently training and testing all models within isolated dataset boundaries (HDFS, Spark, Hadoop) to avoid cross-system contamination.

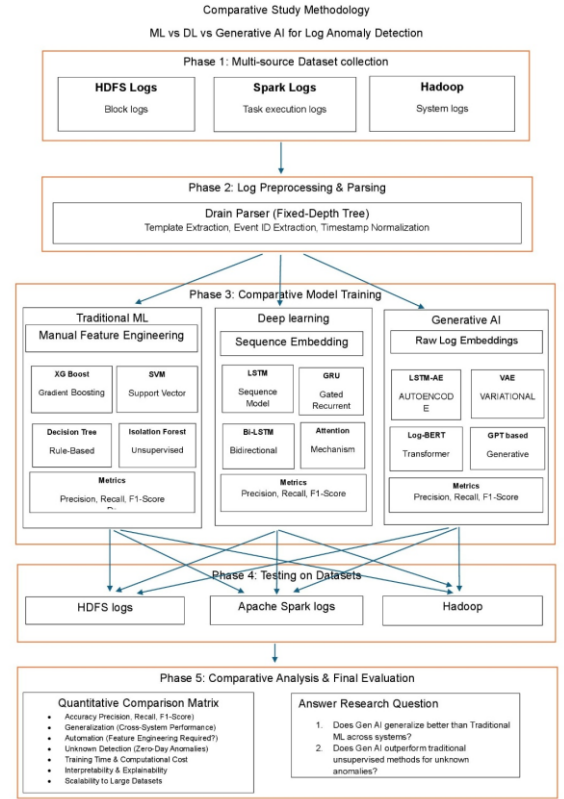


Fig. 1. Five-phase comparative study methodology.

A. Phase 1 – Multi-Source Dataset Collection

HDFS block logs, Apache Spark task execution logs, and Hadoop system logs form a multi-source repository for heterogeneous comparative testing.

B. Phase 2 – Log Preprocessing and Parsing

Raw logs are processed with the Drain parser [3], a fixed-depth tree approach, performing template extraction, event ID extraction, and timestamp normalization.

C. Phase 3 – Comparative Model Architectures

Three parallel pipelines are implemented: (i) Traditional ML (XGBoost, Random Forest, Isolation Forest) using Event Count Matrices; (ii) Deep Learning sequence models (LSTM, GRU, Bi-LSTM with attention, CNN) operating directly on log

sequences; and (iii) Generative AI (VAE, GAN, LSTM-Autoencoder) trained on normal-only data using raw log embeddings, with anomaly scores derived from reconstruction probability.

D. Phase 4 – Training and Testing Dynamics

Each model is evaluated with an 80/20 train-test split sourced from the same dataset (intra-system validation), independently repeated across HDFS, Spark, and Hadoop to verify performance consistency without cross-contaminating training distributions.

E. Phase 5 – Comparative Analysis

Models are assessed via Precision, Recall, F1-score, ROC-AUC, and PR-AUC, alongside qualitative criteria: performance consistency, automation level, zero-day detection capability, training time, interpretability, and scalability.

V. RESULTS AND DISCUSSION

A. HDFS Baseline Performance

Models were evaluated on an 80/20 split of the HDFS dataset (558,223 normal, 16,838 anomaly log vectors; 115,013-instance test set). XGBoost achieved near-perfect performance, with 5 false positives and 1 missed anomaly out of 3,368.

```

Classification Report:

```

	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	111645
1	0.9985	0.9997	0.9991	3368
accuracy			0.9999	115013
macro avg	0.9993	0.9998	0.9995	115013
weighted avg	0.9999	0.9999	0.9999	115013

```

Confusion Matrix:

```

[[111640	5]
[	1 3367]]

Fig. 2. HDFS ML XGBoost confusion matrix.

```

Running shuffle test (performance should drop)...

```

	precision	recall	f1-score	support
0	0.9350	0.2801	0.4311	111645
1	0.0146	0.3545	0.0281	3368
accuracy			0.2823	115013
macro avg	0.4748	0.3173	0.2296	115013
weighted avg	0.9080	0.2823	0.4193	115013

Fig. 3. XGBoost shuffling-test performance.

```

Running 5-fold cross-validation...
F1 per fold: [0.9976 0.9972 0.9985 0.9981 0.9981]
Mean F1: 0.9979 (+/- 0.0005)

```

Fig. 4. XGBoost F1-score summary.

SVM achieved strong baseline performance (128 false positives, 2 missed anomalies), though XGBoost outperformed it across all key parameters, reflecting its superior capacity for non-linear decision boundaries.

```

PS C:\Users\Pc\Desktop\Asad_project> python Train_test_split.
>>
Model training complete.

Classification Report:

```

	precision	recall	f1-score	support
0	1.0000	0.9989	0.9994	111645
1	0.9634	0.9994	0.9811	3368
accuracy			0.9989	115013
macro avg	0.9817	0.9991	0.9902	115013
weighted avg	0.9989	0.9989	0.9989	115013

Fig. 5. SVM results on HDFS.

```

Confusion Matrix:

```

[[111517	128]
[	2 3366]]

Fig. 6. SVM confusion matrix.

```

Running shuffle test (performance should drop)...

```

	precision	recall	f1-score	support
0	0.9398	0.3592	0.5197	111645
1	0.0110	0.2372	0.0211	3368
accuracy			0.3556	115013
macro avg	0.4754	0.2982	0.2704	115013
weighted avg	0.9126	0.3556	0.5051	115013

```

Running 5-fold cross-validation...
F1 per fold: [0.9796 0.9791 0.9769 0.9816 0.9809]
Mean F1: 0.9796 (+/- 0.0016)

```

Fig. 7. SVM results after shuffling.

The supervised Deep Learning pipeline evaluated sequence models without rigid manual feature engineering. LSTM produced 66 false positives and missed 4 anomalies (ROC-AUC = 1.0); GRU performed comparably, missing only 3 anomalies.

```

35997.35997 145.14514514514514
Classification Report:

```

	precision	recall	f1-score	support
0	1.0000	0.9994	0.9997	111645
1	0.9808	0.9988	0.9897	3368
accuracy			0.9994	115013
macro avg	0.9904	0.9991	0.9947	115013
weighted avg	0.9994	0.9994	0.9994	115013

```

Confusion Matrix:
[[111579 66]
 [ 4 3364]]

ROC-AUC: 1.0
PR-AUC: 0.9995

```

Fig. 8. DL LSTM confusion matrix and ROC-AUC results.

```

Classification Report:
      precision    recall  f1-score   support

     0       1.0000      0.9994      0.9997      111645
     1       0.9808      0.9991      0.9899        3368

 accuracy          0.9994      0.9993      0.9994      115013
 macro avg          0.9904      0.9993      0.9948      115013
 weighted avg       0.9994      0.9994      0.9994      115013

Confusion Matrix:
[[111579    66]
 [     3   3365]]

ROC-AUC: 1.0
PR-AUC : 0.9995

Saved models:
best_gru_model.keras
final_gru_model.keras
PS C:\Users\Pc\Desktop\Asad_project>

```

Fig. 9. GRU confusion matrix and ROC-AUC results.

Generative AI models were trained exclusively on normal sequences. The LSTM-Autoencoder produced 1,137 false positives and 398 missed anomalies (ROC-AUC 0.9964, PR-AUC 0.9049); the LSTM-VAE produced 1,101 false positives and 866 missed anomalies (ROC-AUC 0.9876, PR-AUC 0.7854).

```

Classification Report:
      precision    recall  f1-score   support

     0       0.9964      0.9898      0.9931      111645
     1       0.7232      0.8818      0.7946        3368

 accuracy          0.9867      0.9867      0.9867      115013
 macro avg          0.8598      0.9358      0.8939      115013
 weighted avg       0.9884      0.9867      0.9873      115013

Confusion Matrix:
[[110508   1137]
 [    398   2970]]

ROC-AUC: 0.9964
PR-AUC : 0.9049

Saved files:
best_lstm_autoencoder.keras
final_lstm_autoencoder.keras
lstm_autoencoder_train_errors.npy
lstm_autoencoder_test_errors.npy
lstm_autoencoder_metadata.pkl

```

Fig. 10. Generative AI LSTM-Autoencoder confusion matrix and ROC-AUC results.

```

Classification Report:
      precision    recall  f1-score   support

     0       0.9922      0.9901      0.9912      111645
     1       0.6944      0.7429      0.7178        3368

 accuracy          0.9829      0.9829      0.9829      115013
 macro avg          0.8433      0.8665      0.8545      115013
 weighted avg       0.9835      0.9829      0.9832      115013

Confusion Matrix:
[[110544   1101]
 [    866   2502]]

ROC-AUC: 0.9876
PR-AUC : 0.7854

Saved files:
lstm_vae_encoder.keras
lstm_vae_decoder.keras
lstm_vae_train_errors.npy
lstm_vae_test_errors.npy
lstm_vae_metadata.pkl

```

Fig. 11. Generative AI LSTM-VAE confusion matrix and ROC-AUC results.

Table I and Fig. 12 summarize the HDFS comparison: XGBoost and the supervised DL models (LSTM, GRU) dominate, while generative models establish a lower, but non-trivial, zero-day detection baseline.

TABLE I
HDFS MODEL COMPARISON

Model	Type	Prec.	Recall	F1	ROC-AUC
XGBoost	Trad. ML	99.85%	99.97%	99.91%	—
SVM	Trad. ML	96.34%	99.94%	98.11%	—
LSTM	Sup. DL	98.08%	99.88%	98.97%	100.0%
GRU	Sup. DL	98.08%	99.91%	98.99%	100.0%
LSTM-AE	Generative	72.32%	88.18%	79.46%	99.64%
LSTM-VAE	Generative	69.44%	74.29%	71.78%	98.76%

Comparison so far						
Model	Type	Anomaly Precision	Anomaly Recall	Anomaly F1	ROC-AUC	PR-AUC
LSTM	Supervised DL	98.08%	99.88%	98.97%	100.00%	99.95%
GRU	Supervised DL	98.08%	99.91%	98.99%	100.00%	99.95%
LSTM Autoencoder	Generative / Normal-only	72.32%	88.18%	79.46%	99.64%	90.49%
LSTM VAE	Generative / Normal-only	69.44%	74.29%	71.78%	98.76%	78.54%

Fig. 12. HDFS comparison matrix across all evaluated models.

B. Spark Logs Analysis

On Apache Spark logs, Random Forest achieved the strongest ROC-AUC (0.9902), while Isolation Forest (0.9559) and One-Class SVM (0.2388) performed poorly; Isolation Forest also suffered low precision and F1 (0.04).

```

Train/Test split kar rahe hain...
X_train: (1329464, 3)
X_test: (332366, 3)

Train anomalies: 1820
Test anomalies: 455

SMOTE apply kar rahe hain...

After SMOTE:
X_train shape: (2655288, 3)
Normal: 1,327,644
Anomaly: 1,327,644

```

Fig. 13. Spark ML train and test results.

```

1. Isolation Forest training...
ROC-AUC: 0.9559 ✓
2. Random Forest training...
ROC-AUC: 0.9902 ✓
3. SGD One-Class SVM training...
ROC-AUC: 0.2388 ✓

=====
FINAL RESULTS COMPARISON
=====

Isolation Forest – ROC-AUC: 0.9559
precision    recall  f1-score   support

   Normal     1.00     1.00     1.00   331911
  Anomaly     0.04     0.04     0.04     455

 accuracy          1.00   332366
macro avg          0.52   332366
weighted avg       1.00   332366

Random Forest – ROC-AUC: 0.9902
precision    recall  f1-score   support

...
 accuracy          1.00   332366
macro avg          0.50   332366
weighted avg       1.00   332366

```

Fig. 14. Isolation Forest and Random Forest results.

The LSTM-Autoencoder reached ROC-AUC 0.9822 with recall 0.93 for anomalies, but precision of only 0.02 ($F1 = 0.05$), reflecting severe false-positive volume.

```

Mounted at /content/drive
TensorFlow version: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Data load ho raha hai...
Shape: (1661830, 6)

X_train: (1329464, 3)
X_test: (332366, 3)
Train anomalies: 1,820
Test anomalies: 455

Sab ready hai! ✓

```

Fig. 15. Deep Learning train and test results (Spark).

```

=== LSTM Autoencoder Results ===
precision    recall  f1-score   support

   Normal     1.00     0.95     0.97   331911
  Anomaly     0.02     0.93     0.05     455

 accuracy          0.95   332366
macro avg          0.51   332366
weighted avg       1.00   332366

ROC-AUC Score: 0.9822

Confusion Matrix:
[[315349 16562]
 [   31  424]]

```

Fig. 16. LSTM results (Spark).

The CNN classifier recorded the highest overall ROC-AUC (0.9978) with perfect recall (1.00), but likewise suffered severe precision collapse (0.09), yielding $F1 = 0.16$.

```

=== CNN Results ===
precision    recall  f1-score   support

   Normal     1.00     0.99     0.99   331911
  Anomaly     0.09     1.00     0.16     455

 accuracy          0.99   332366
macro avg          0.54   332366
weighted avg       1.00   332366

ROC-AUC Score: 0.9978

Confusion Matrix:
[[327261 4650]
 [    2  453]]

```

Fig. 17. CNN results (Spark).

```

=====
DEEP LEARNING vs TRADITIONAL ML – FULL COMPARISON
=====

Model          ROC-AUC    Recall    F1
-----
Random Forest    0.9902    0.89    0.44
Isolation Forest 0.9559    0.04    0.04
One-Class SVM    0.2388    0.01    0.00
LSTM Autoencoder 0.9822    0.93    0.05
CNN Classifier   0.9978    TBD     TBD

```

Fig. 18. Deep Learning vs Traditional ML comparison (Spark).

Both generative models performed poorly on Spark: the VAE reached ROC-AUC 0.7700 with $F1 = 0.01$, while the GAN failed to detect anomalies effectively (ROC-AUC 0.2479, $F1 = 0.01$).

```

Mounted at /content/drive
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Data load ho raha hai...
Shape: (1661830, 6)

X_train: (1329464, 3)
X_test: (332366, 3)
Normal train samples: 1,327,644

Sab ready hai! ✓

```

Fig. 19. Generative AI train and test results (Spark).

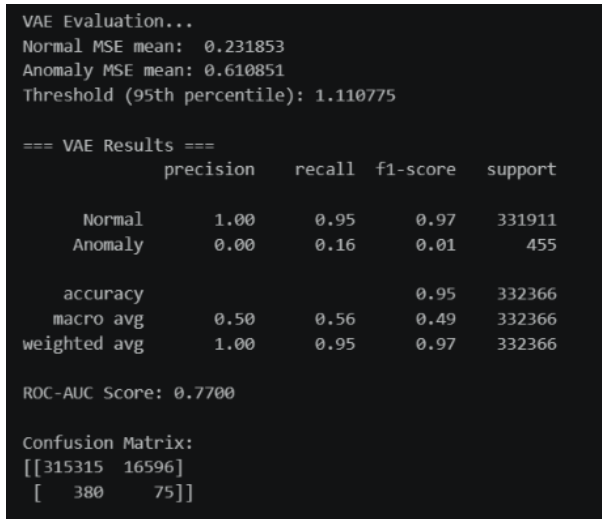


Fig. 20. VAE results (Spark).

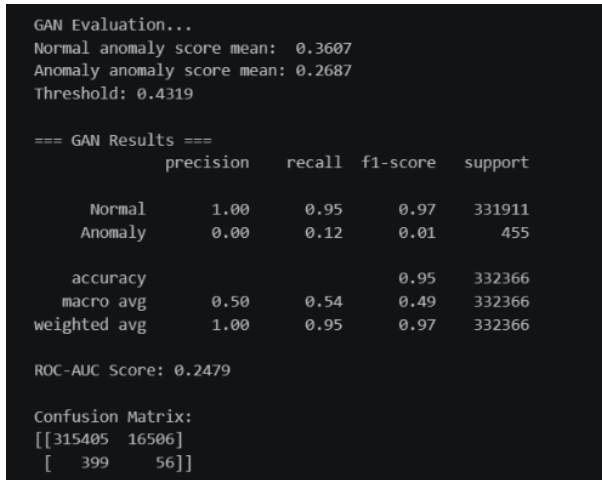


Fig. 21. GAN results (Spark).

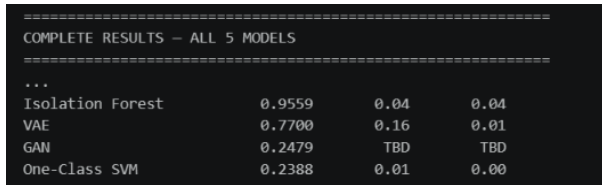


Fig. 22. Generative model summary panel (Spark).

Spark Comparative Analysis Synthesis

The complex, overlapping nature of Spark execution logs created a massive false-positive problem: the LSTM-Autoencoder alone generated 16,562 false positives, dropping F1 to 0.05 despite a strong ROC-AUC of 0.9822. Both generative models collapsed to $F1 = 0.01$, indicating that Spark's structural noise and concurrency fundamentally disrupt latent-distribution mapping.

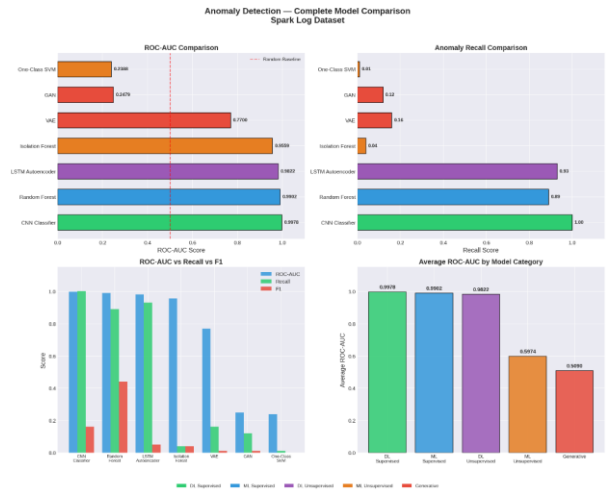


Fig. 23. Spark logs model comparison.

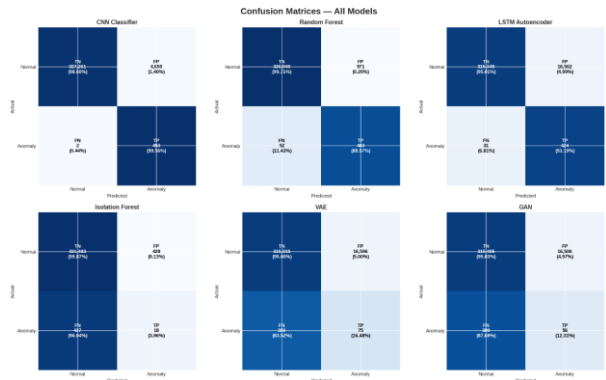


Fig. 24. Spark logs confusion matrix.



Fig. 25. Spark logs comparative summary.

C. Hadoop Logs Analysis

On Hadoop, Random Forest achieved perfect metrics (ROC-AUC, Recall, F1 = 1.0000); this is attributable to per-application label leakage rather than genuine separability, making the Spark results more reliable for cross-dataset comparison. Isolation Forest and One-Class SVM performed poorly (ROC-AUC 0.2092 and 0.2383).

```

Mounted at /content/drive
Loading Hadoop data...
Shape: (393433, 7)

Label Distribution:
label
1    368148
0    25285
Name: count, dtype: int64

SMOTE apply ho raha hai...

After SMOTE:
Normal: 294,518
Anomaly: 294,518
X_test: (78687, 3)

Sab ready hail! ✓

```

Fig. 26. Hadoop ML label-logs distribution.

```

...
=====
1. Isolation Forest training...
ROC-AUC: 0.2092 ✓
2. Random Forest training...
ROC-AUC: 1.0000 ✓
3. SGD One-Class SVM training...
ROC-AUC: 0.2383 ✓
=====
HADOOP - TRADITIONAL ML RESULTS
=====

Isolation Forest - ROC-AUC: 0.2092
precision recall f1-score support

Normal    0.00    0.01    0.00    5057
Anomaly    0.87    0.46    0.60    73630

accuracy          0.43    78687
macro avg         0.44    0.24    0.30    78687
weighted avg      0.82    0.43    0.57    78687

Random Forest - ROC-AUC: 1.0000
precision recall f1-score support

...
accuracy          0.21    78687
macro avg         0.42    0.26    0.18    78687
weighted avg      0.76    0.21    0.30    78687

```

Fig. 27. ML Isolation Forest and Random Forest results (Hadoop).

Deep learning performance was mixed on Hadoop. The CNN classifier performed strongly (ROC-AUC 0.9617, F1 = 0.91); the LSTM-Autoencoder degraded sharply relative to its Spark result (ROC-AUC 0.6519, F1 = 0.45).

```

Mounted at /content/drive
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Data load ho raha hai...
Shape: (393433, 7)

X_train: (314746, 3)
X_test: (78687, 3)
Normal train: 20,228
Test anomalies: 73,630

Sab ready hail! ✓

```

Fig. 28. Deep Learning train/test results (Hadoop).

```

=====
HADOOP - DEEP LEARNING RESULTS
=====

LSTM Autoencoder - ROC-AUC: 0.6519
precision recall f1-score support

Normal    0.08    0.95    0.15    5057
Anomaly    0.99    0.29    0.45    73630
...
accuracy          0.85    78687
macro avg         0.65    0.91    0.69    78687
weighted avg      0.95    0.85    0.88    78687

```

Fig. 29. LSTM-Autoencoder results (Hadoop).

```

... Saved! ✓
{
  "CNN_Classifier": {
    "ROC_AUC": 0.9617,
    "Recall": 0.84,
    "F1": 0.91
  },
  "LSTM_Autoencoder": {
    "ROC_AUC": 0.6519,
    "Recall": 0.29,
    "F1": 0.45
  }
}

```

Fig. 30. CNN and LSTM comparison (Hadoop).

Generative models remained weak on Hadoop: the GAN achieved ROC-AUC 0.6133 (F1 = 0.03), while the VAE recorded ROC-AUC 0.5235 (F1 = 0.12).

```

Mounted at /content/drive
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Data load ho raha hai...
Shape: (393433, 7)

X_train: (314746, 3)
X_test: (78687, 3)
Normal train: 20,228

Sab ready hail! ✓

```

Fig. 31. Generative AI train/test results (Hadoop).

```

=====
HADOOP - GENERATIVE MODELS RESULTS
=====

VAE - ROC-AUC: 0.5235
precision recall f1-score support

Normal    0.07    0.95    0.12    5057
Anomaly    0.95    0.07    0.12    73630

accuracy          0.12    78687
macro avg         0.51    0.51    0.12    78687
weighted avg      0.89    0.12    0.12    78687

GAN - ROC-AUC: 0.6133
precision recall f1-score support

...
accuracy          0.07    78687
macro avg         0.44    0.48    0.07    78687
weighted avg      0.76    0.07    0.03    78687

```

Fig. 32. VAE and GAN results (Hadoop).

The summary confirms Random Forest's absolute superiority on Hadoop, recording zero false positives and zero false negatives; the CNN also demonstrated high accuracy with only 50 false positives among tens of thousands of records.

```
=====
HADOOP — ALL MODELS COMPLETE SUMMARY
=====
```

Model	ROC-AUC	Recall	F1
Random Forest	1.0000	1.00	1.00
CNN Classifier	0.9617	0.84	0.91
LSTM Autoencoder	0.6519	0.29	0.45
GAN	0.6133	0.01	0.03
VAE	0.5235	0.07	0.12
One-Class SVM	0.2383	0.20	0.32
Isolation Forest	0.2092	0.46	0.60

Fig. 33. Hadoop — all-model results summary.

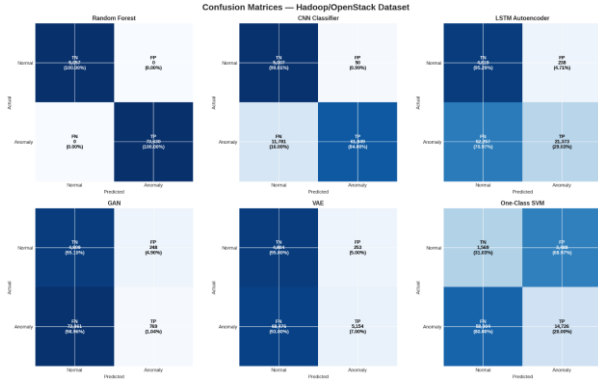


Fig. 34. Hadoop — all-model confusion matrix.

D. Overall Performance Synthesis

Table II summarizes F1-scores across all three datasets. Random Forest is the most consistently reliable model overall; CNN shows excellent ROC-AUC but highly dataset-dependent F1 (0.16 on Spark vs. 0.91 on Hadoop); Generative and unsupervised models remain consistently weak across environments.

TABLE II
OVERALL MODEL F1-SCORES ACROSS DATASETS

Model	Category	HDFS	Spark	Hadoop
XGBoost	Trad. ML (S)	0.9991	—	—
SVM	Trad. ML (S)	0.9811	—	—
Random Forest	Trad. ML (S)	—	0.44	1.00
LSTM	DL (Sup.)	0.9897	—	—
GRU	DL (Sup.)	0.9899	—	—
CNN	DL (Sup.)	—	0.16	0.91
Isolation Forest	Trad. ML (U)	—	0.04	0.60
One-Class SVM	Trad. ML (U)	—	0.00	0.32
LSTM-AE	Generative	0.7946	0.05	0.45
VAE	Generative	0.7178	0.01	0.12
GAN	Generative	—	0.01	0.03

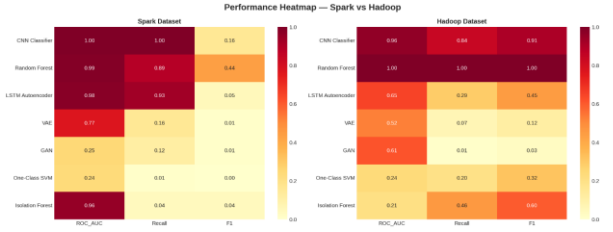


Fig. 35. Overall performance matrix — Spark vs. Hadoop across all evaluated models.

Three key patterns emerge. First, the "HDFS illusion": nearly every model performs exceptionally well on HDFS, suggesting the anomaly-detection problem is largely solved—an impression that does not generalize. Second, the Spark reality check: the complex, overlapping nature of Spark execution logs collapses F1-scores below 0.50 for almost every model despite strong ROC-AUC values. Third, the generative struggle: while theoretically advantageous for removing manual feature engineering, Generative AI consistently underperforms across all environments, peaking at F1 = 0.7946 on HDFS and failing on Spark and Hadoop.

VI. CONCLUSION AND FUTURE WORK

This paper presented a comparative study of Traditional ML, Deep Learning, and Generative AI for automated log anomaly detection across HDFS, Apache Spark, and Hadoop. The findings reveal a clear trade-off between accuracy and functional autonomy: traditional supervised classifiers achieve near-perfect F1-scores on structurally consistent datasets but depend on manual feature engineering; supervised deep sequence models remove this dependency while matching accuracy, yet degrade under Spark's noise and concurrency; and Generative AI, despite consistently high ROC-AUC, achieves lower F1-scores overall, establishing only a baseline capability for zero-day anomaly discovery rather than outright superiority over supervised approaches.

Future work will explore dynamic thresholding (e.g., extreme value theory) to reduce false positives in generative models, online incremental learning for concept-drift mitigation, fine-grained parsing of variable log fields, and hybrid neuro-symbolic frameworks combining ensemble robustness with generative flexibility.

ACKNOWLEDGMENT

The author thanks Prof. Dr. Imran Ihsan for his supervision and guidance throughout this research.

REFERENCES

- [1] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in Proc. IEEE 34th Int. Symp. Software Reliability Engineering (ISSRE), 2023, pp. 355–366.
- [2] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI), 2019, pp. 4739–4745.
- [3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in Proc. IEEE Int. Conf. Web Services (ICWS), 2017, pp. 33–40.
- [4] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in Proc. ACM

SIGSAC Conf. Computer and Communications Security, 2017, pp. 1285–1298.

- [5] X. Zhang et al., "Onion: Identifying incident-indicating logs for cloud systems," in Proc. 29th ACM Joint Meeting European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE), 2021, pp. 1253–1263.
- [6] W. Xu, L. Huang, A. Fox, D. A. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in Proc. ACM SIGOPS 22nd Symp. Operating Systems Principles (SOSP), 2009, pp. 117–132.
- [7] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.
- [8] Q. Lin, H. Zhang, J. G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in Proc. 38th Int. Conf. Software Engineering Companion (ICSE-C), 2016, pp. 102–111.